3. Multimodal Visual Understanding + Robotic Arm Gripping (Voice Version)

1. Course Content

1. Learn how the multimodal large model integrates speech recognition, 2D visual perception, spatial geometric reasoning, and 3D inverse-kinematics control of the robotic arm;
2. Through voice dialogue interaction, experience the large model autonomously recognizing target objects, computing their relative spatial positions, and controlling a 6-DOF robotic arm to perform precise spatial grasping (`pick`);
3. Learn how the large model reasons about the relative pose between targets and completes relative placement (`place`);
4. Learn AprilTag-based 3D elevation perception filtering and automated sorting removal (`apriltag_remove_higher`);
5. Deeply analyze the underlying spatial coordinate solving logic of the embodied large model's decision layer and the action service side.
6. **This case uses the matching 3 * 3 * 3 blocks and 3 * 3 * 3 machine-code wooden blocks**

---

2. Preparation

2.1 Description

> This lesson is applicable to controllers such as Raspberry Pi 5, Jetson Orin NANO, and D-Robotics RDK X5:
>
> - **Raspberry Pi 5**: You need to first enter the Docker container before executing ROS 2 related commands. Refer to [3.Enter Docker (For Raspberry Pi)] in [Robot Configuration and Operation Guide];
> - **Orin / RDK X5**: Directly run the commands in the vehicle terminal; there is no need to enter Docker.

2.2 Start the Dify Local Large Model Environment

Enter the following command in the host terminal to start the Dify service:

```
sh ~/bringup_dify.sh
```

The terminal prints the following information, indicating that the Dify service is connected successfully:



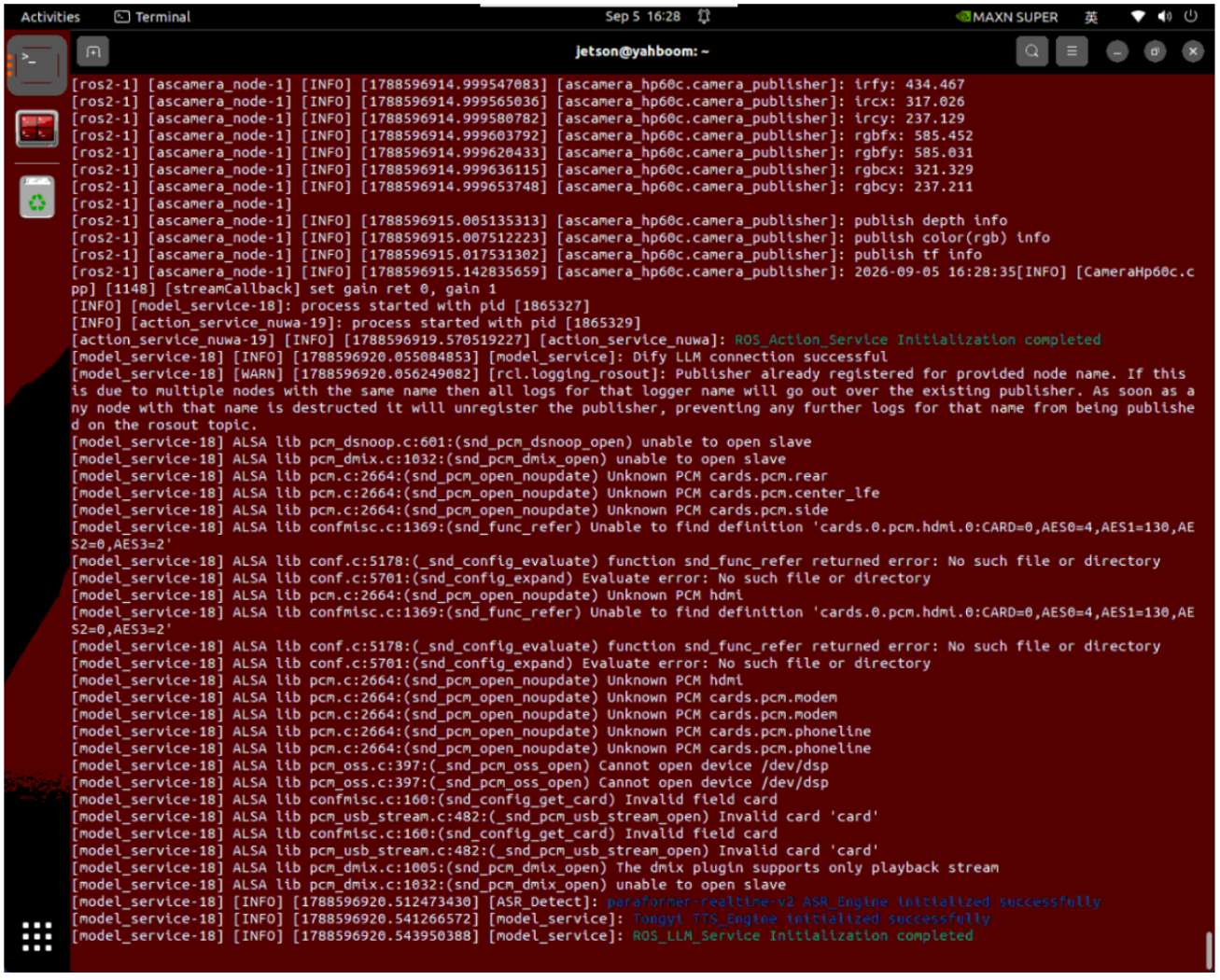
3. Running the Cases

3.1 Start the Embodied Intelligence Master Control Service (Voice Mode)

Open a terminal on the vehicle and enter the following command to start the large model embodied control hub (microphone voice interaction mode is enabled by default):

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After the system initialization is complete, the action service and multimodal perception nodes are ready:



3.2 Voice Interaction Specification

Voice interaction does not require opening an additional text interaction terminal; simply talk through the microphone:

1. First clearly call the wake word: **"Hi,yahboom"** (or "yahboom, yahboom");
2. Listen for the car's voice response: **"I'm here, please give me instructions"** or various personalized wake-up voices;
3. After the response ends, you can speak the test instruction to the robot.

```
model_service-18] [INFO] [1788596920.512473430] [ASR_Detect]: paraformer-realtime-v2 ASR_Engine initialized successfully
model_service-18] [INFO] [1788596920.541266572] [model_service]: Tongyi TTS_Engine initialized successfully
model_service-18] [INFO] [1788596920.543950388] [model_service]: ROS_LLM_Service Initialization completed
model_service-18] [INFO] [1788596939.760860151] [ASR_Detect]: I'm here
action_service_nuwa-19] [INFO] [1788596939.765808185] [action_service_nuwa]: wakeup received
model_service-18] [INFO] [1788596942.020545484] [ASR_Detect]: 1-1-1
model_service-18] [INFO] [1788596942.022199639] [ASR_Detect]: 1-1-1
model_service-18] [INFO] [1788596942.023717111] [ASR_Detect]: 1-1-1
model_service-18] [INFO] [1788596942.026509410] [ASR_Detect]: 1-1-1
model_service-18] [INFO] [1788596942.029065624] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.031017628] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.040286823] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.041426631] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.042472927] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.071567948] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.161998907] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.252488814] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.343293884] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.439410313] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.528417472] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.619597806] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.712022212] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.802312647] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.893573916] [ASR_Detect]: ----
model_service-18] [INFO] [1788596942.983737684] [ASR_Detect]: ----
model_service-18] [INFO] [1788596943.074900441] [ASR_Detect]: ----
model_service-18] [INFO] [1788596943.165869900] [ASR_Detect]: ----
model_service-18] [INFO] [1788596943.257114389] [ASR_Detect]: ----
model_service-18] [INFO] [1788596943.348248982] [ASR_Detect]: ----
model_service-18] [INFO] [1788596943.409160046] [ASR_Detect]: ----
```

3.3 Classic Test Cases

Recommended props for testing:

- Basic block test: 30 × 30 × 30 mm building blocks (solid color blocks such as red, green, yellow, and blue);
- Elevation and tag test: 30 × 30 × 30 mm building blocks affixed with AprilTag codes (they can be stacked in multiple layers to form different heights).

Case 1: Target Object Recognition and Precise Gripping

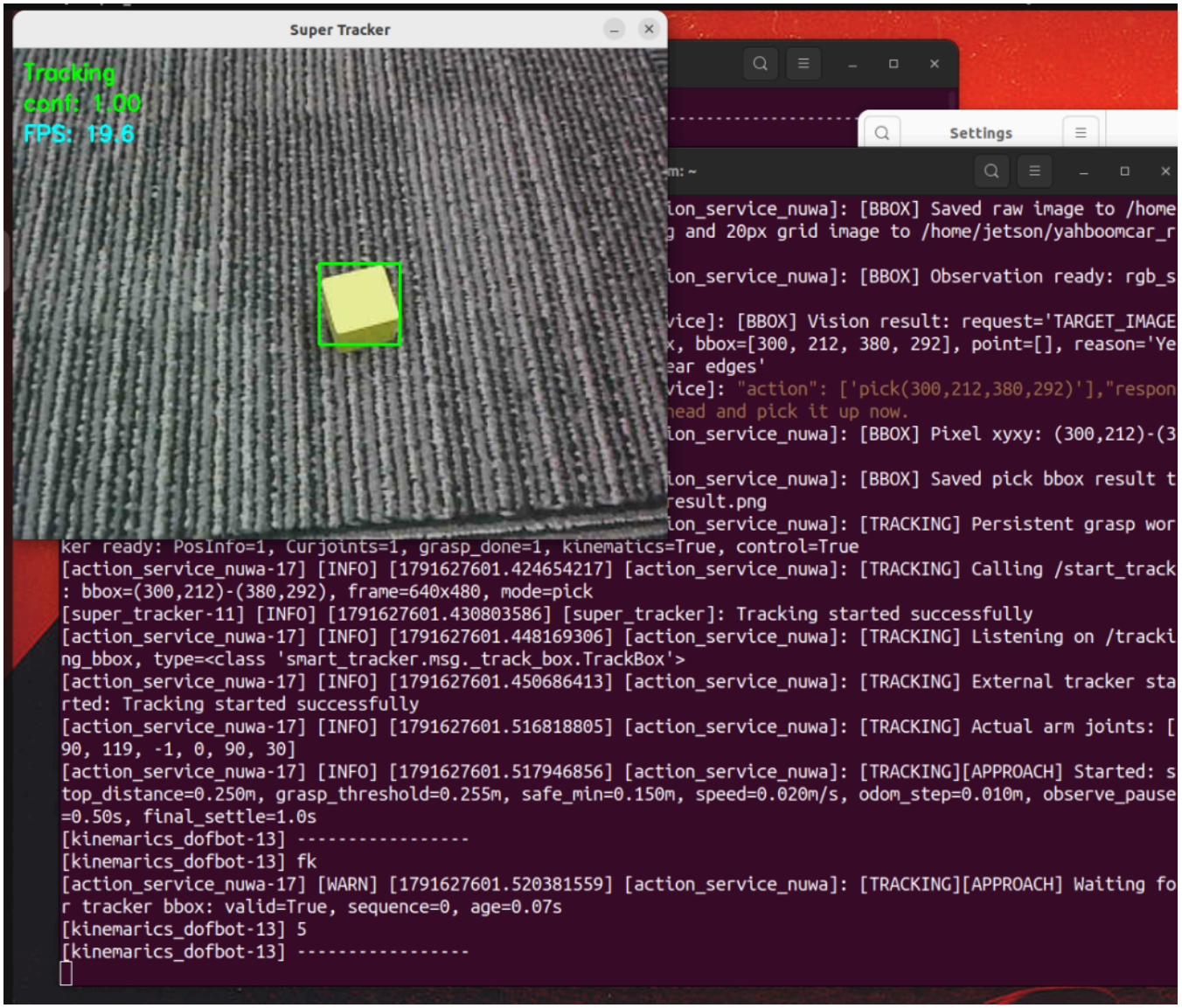
1. After waking up yahboom, speak the test instruction to the car:

   ```
   Find the yellow block in front of you and pick it up.
   ```

2. The terminal prints the steps planned by the large model and the bounding box parameters. The save path of the photo with the boxed recognition result is: `~/yahboomcar_ros2_ws/yahboomcar_ws/cache/bbox_result.png`

Execution Process and Phenomena:

1. After receiving the voice instruction, the large model first issues the `seewhat` instruction to obtain the current color image from the camera;
2. The multimodal model precisely locates the pixel bounding box coordinates `[x1, y1, x2, y2]` of the yellow block and assembles the `pick` action;
3. The robot executes the underlying spatial inverse solution, automatically adjusts the robotic arm angle and opens the gripper, and swoops down to grasp the target block;
4. After a successful grasp, the robotic arm lifts and maintains the current holding pose, and a voice announcement reports that the grasp is complete:



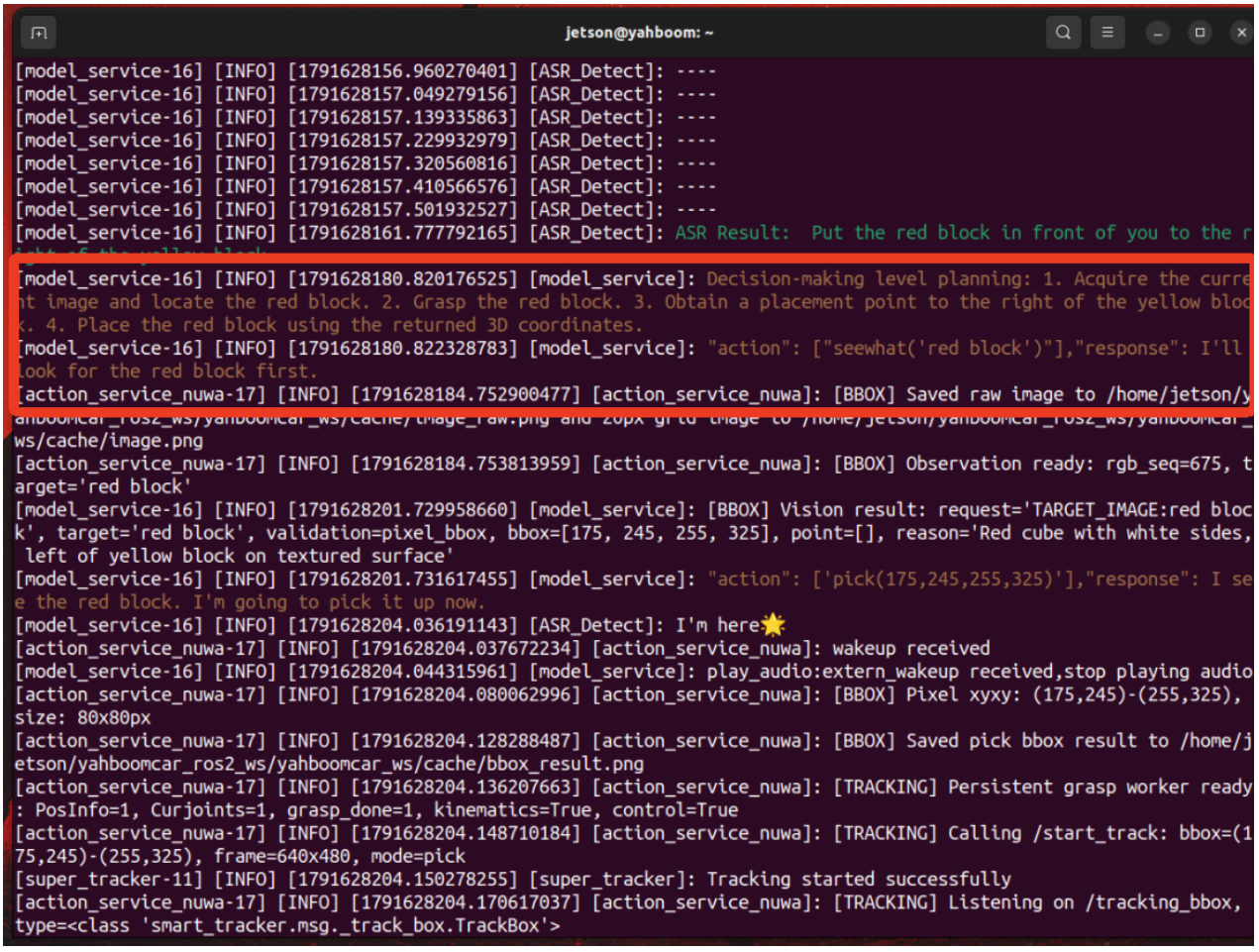
> **Note**
>
> **Robotic arm reset methods**:
>
> - **Method 1**: After waking up yahboom, say: `"Put down the block that was just picked up"`, and the robotic arm will put the block down at a safe position;
> - **Method 2**: After waking up yahboom, say: `"Go take a rest"` (or `"End the current task"`), and the current task cycle ends.

Case 2: Relative Spatial Relationship Placement Between Targets

1. Place a red block and a yellow block on the tabletop;
2. After waking up yahboom, speak the test instruction to the car:

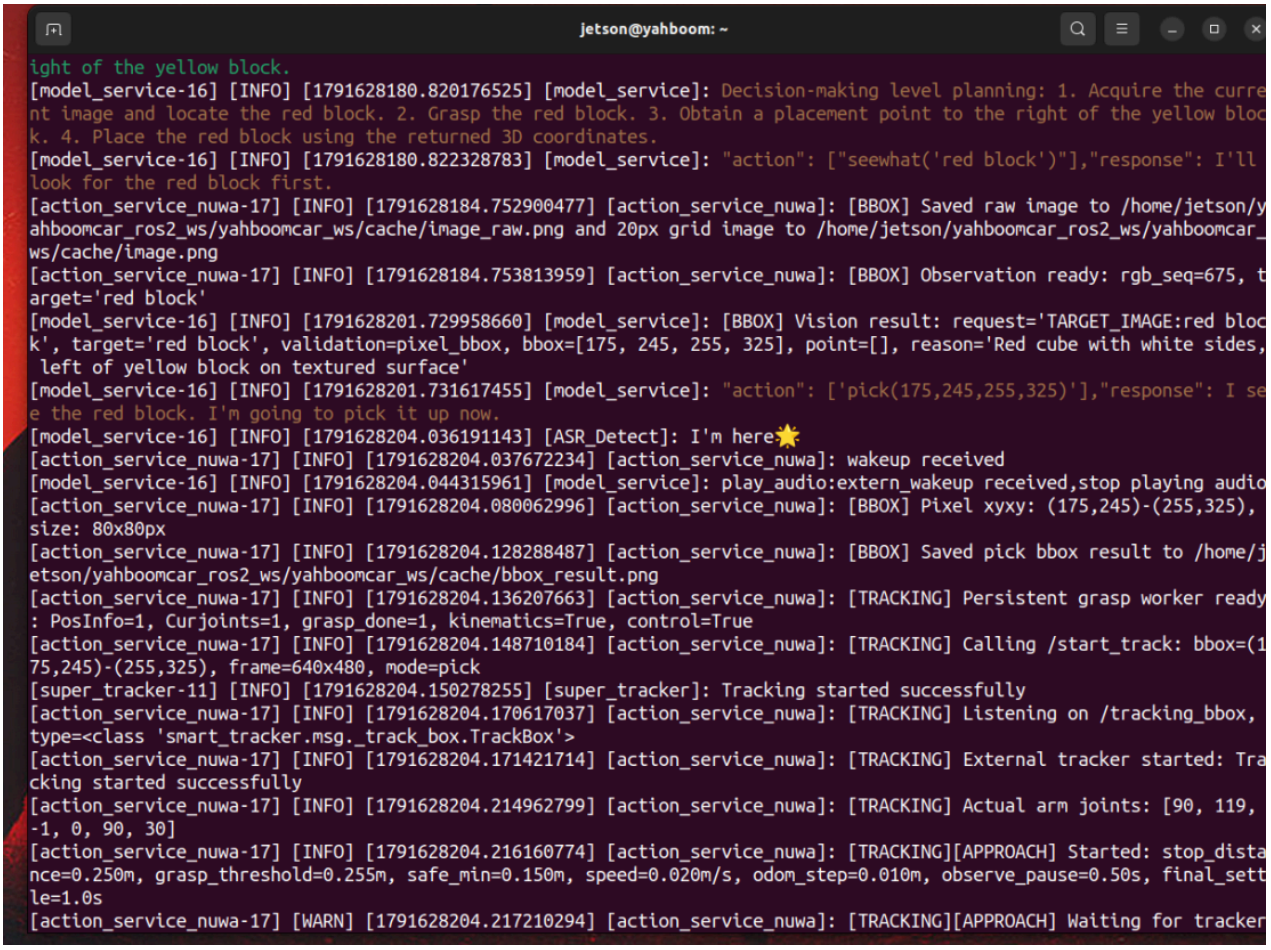
```
Put the red block in front of you to the right of the yellow block.
```

3. The terminal prints the 4-stage task plan parsed by the large model:



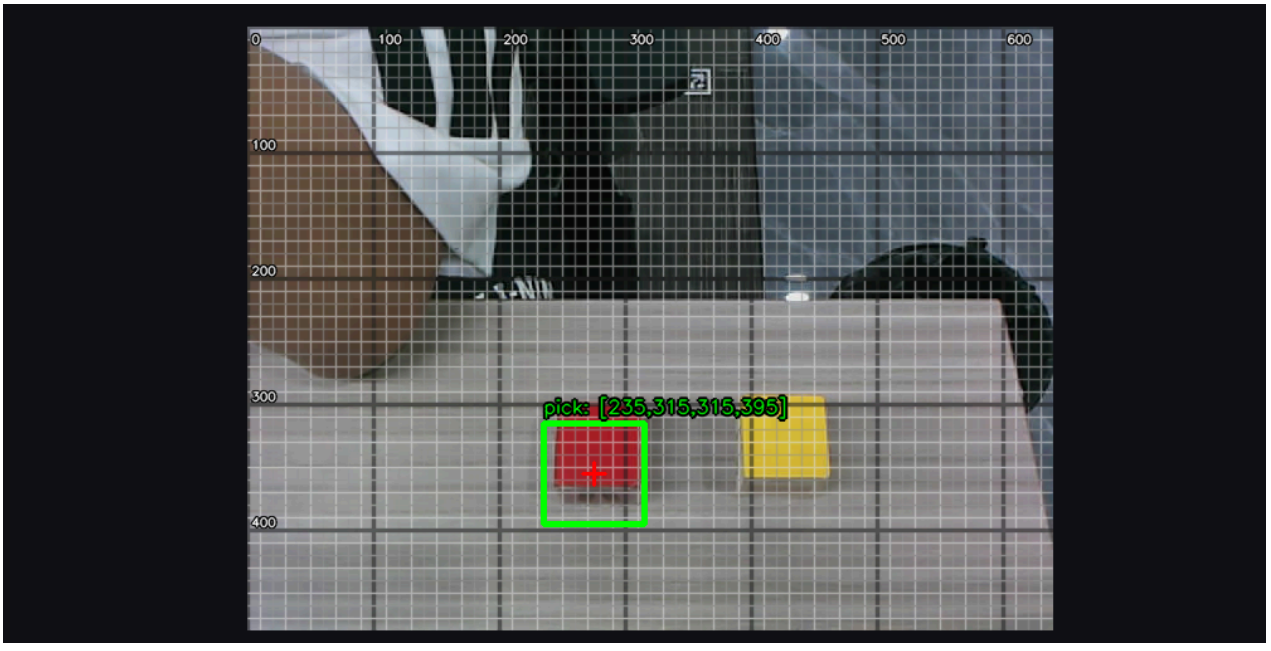
Execution Process and Phenomena:

1. The multimodal large model simultaneously detects the pixel coordinates of the "red block" and the "yellow block" in the current field of view;
2. The robot first calls `pick` to grasp the red block in front of it:

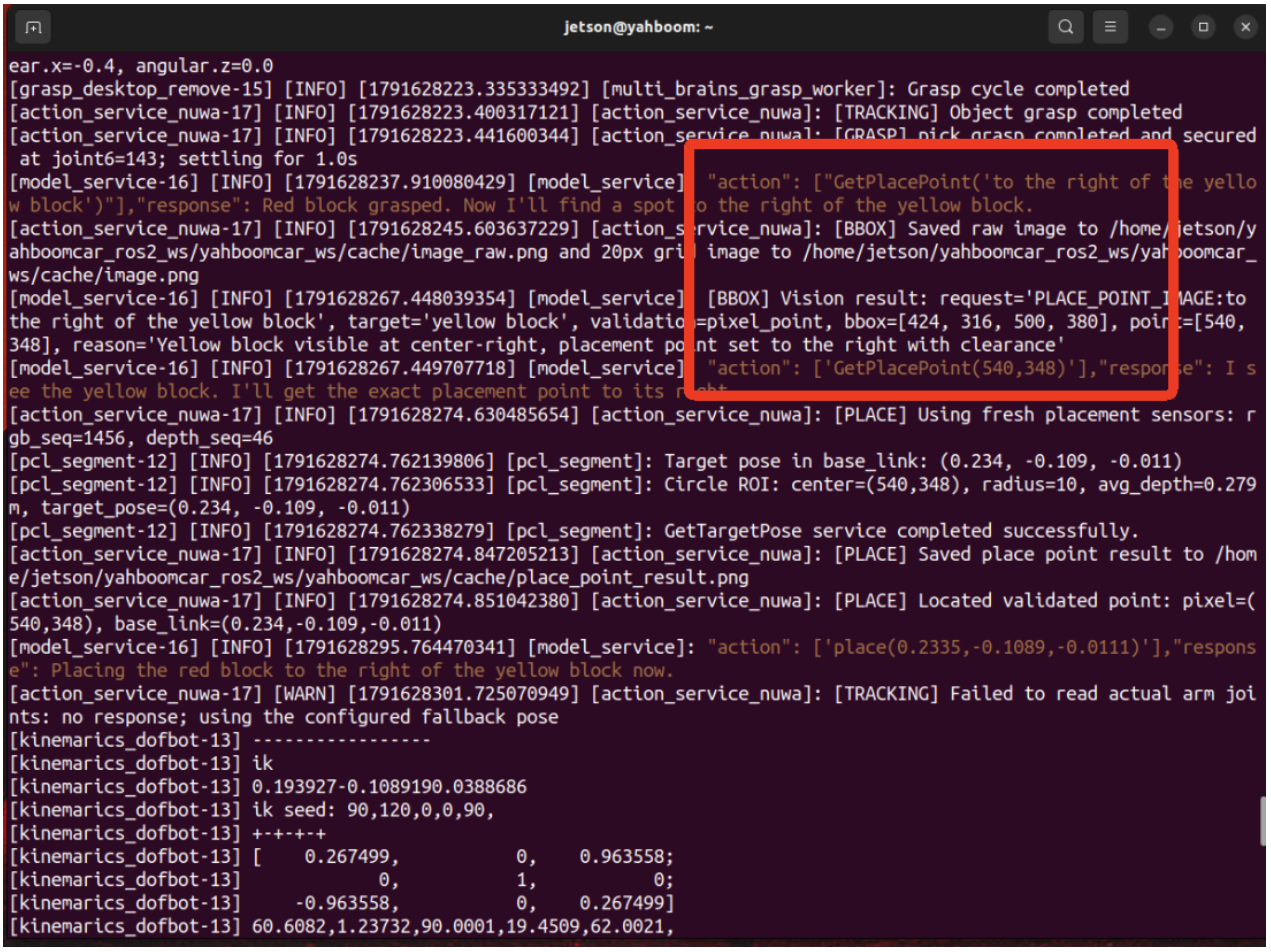


The detection and localization image is saved at:

`~/yahboomcar_ros2_ws/yahboomcar_ws/cache/bbox_result.png`

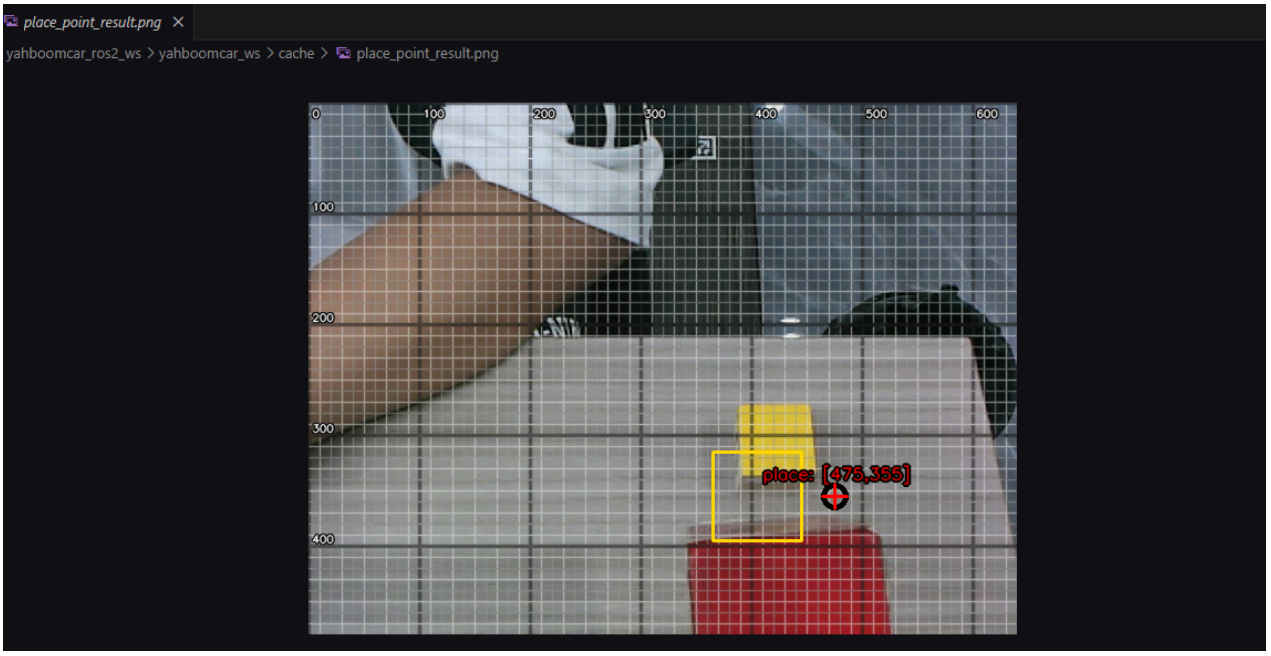


3. Next, based on the coordinates of the yellow block and the geometric topological relationship of "on the right", the large model calls the `place` instruction; the robotic arm rotates to the right side of the base, safely releases the gripper, completes precise positioning, and gives a voice prompt that the task is complete:



The spatial drop-point calculation diagram is saved at:

`~/yahboomcar_ros2_ws/yahboomcar_ws/cache/place_point_result.png`



4. After the task is completed, wake up yahboom and say `"Go take a rest"` to reset the robotic arm and start a new round of interaction:

Case 3: 3D Elevation Perception and Automated Sorting Removal

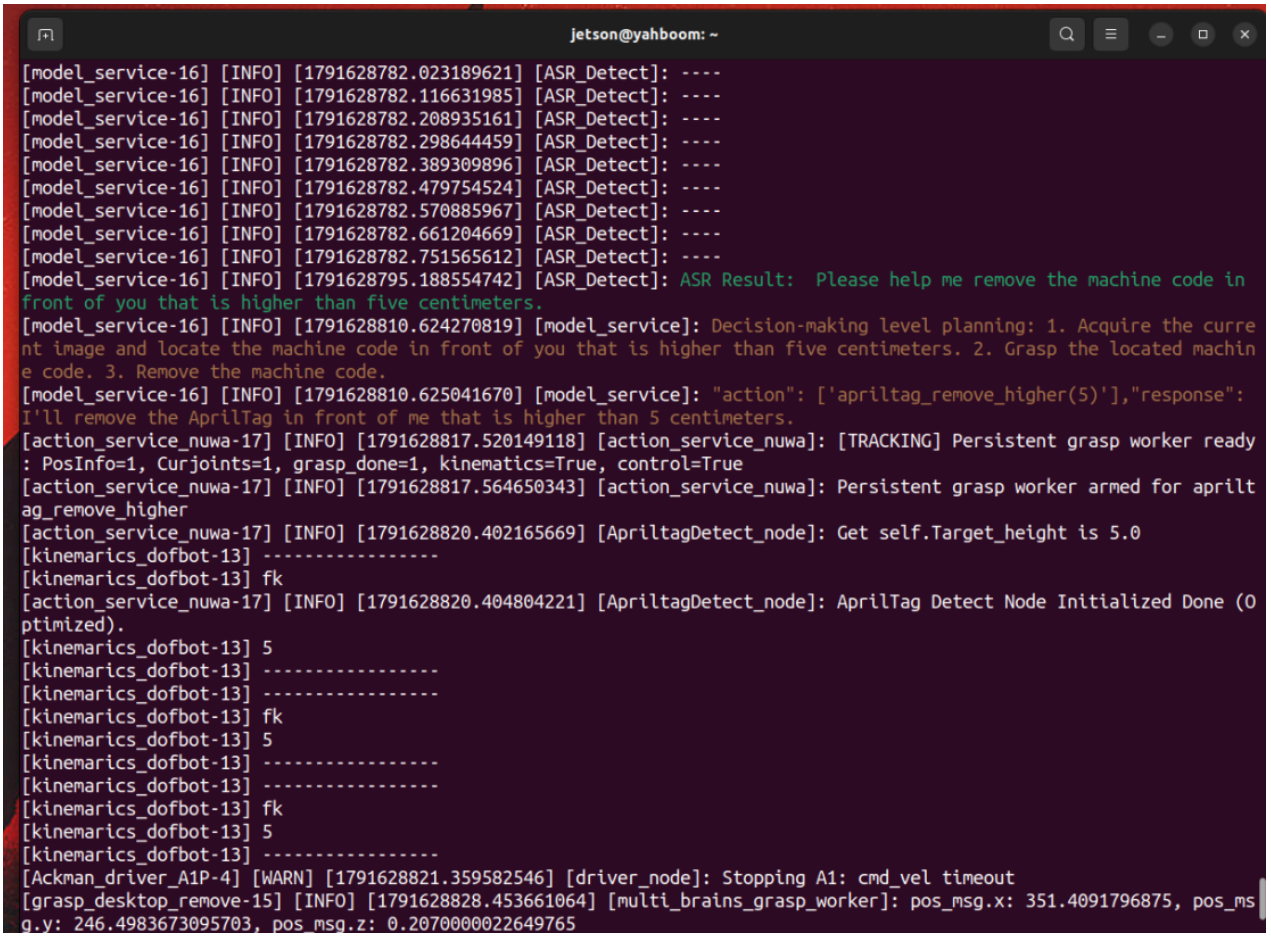
1. Place several wooden blocks with AprilTag tags in the field of view, and raise one of the blocks to more

than 5 centimeters;

2. After waking up yahboom, speak the test instruction to the car:

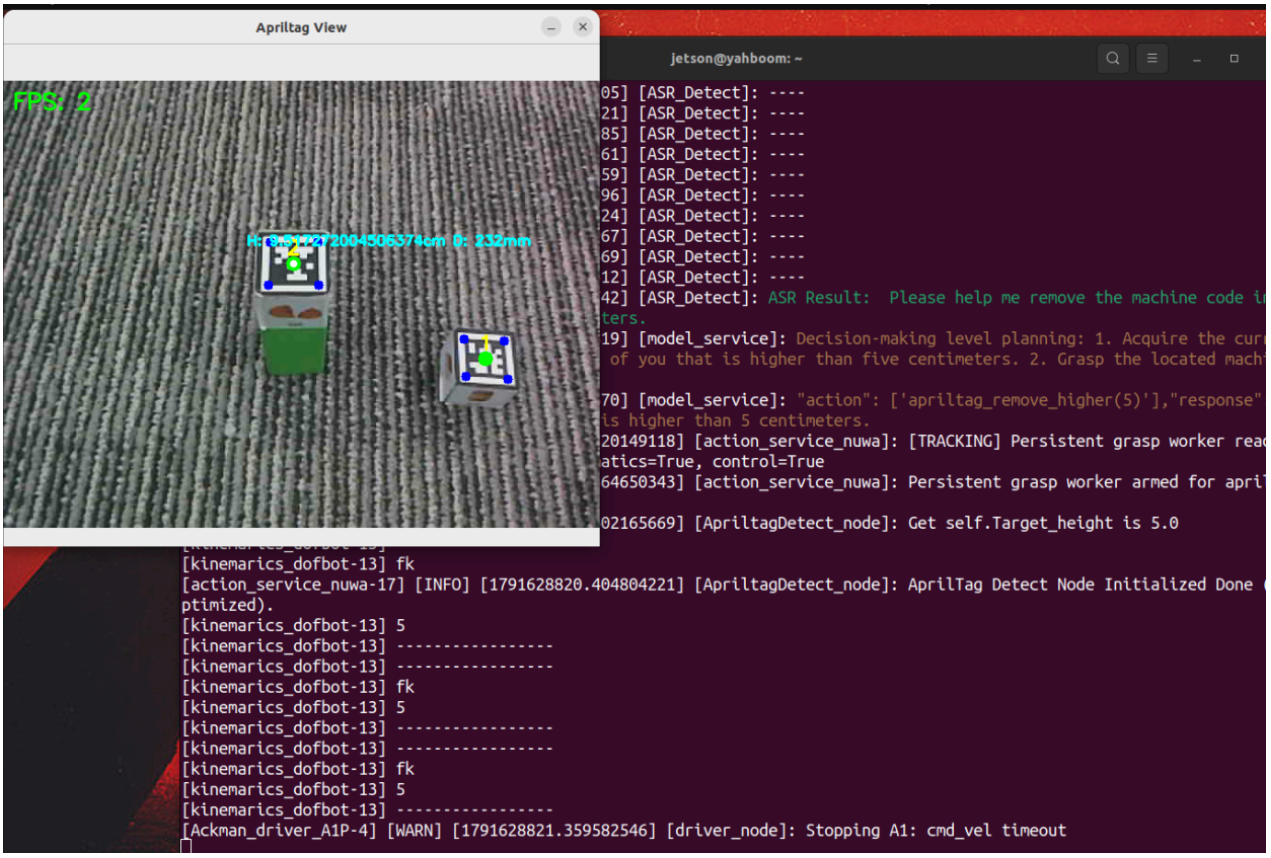
```
Please help me remove the machine code in front of you that is higher than 5
centimeters.
```

3. The terminal prints the `apriltag_remove_higher` action parsed by the large model:



Execution Process and Phenomena:

1. The robot calls the camera to perceive the pose and spatial height of the AprilTag tags, and the window marks the 3D relative height of each tag in real time;
2. When a tag with a height exceeding 5 centimeters (0.05 m) is detected, the robotic arm automatically locates its spatial center and reaches down to grasp it:



3. To put down the grasped machine code, wake up yahboom and say to the car `"Put down the machine code in your hand"`:

---

4. Core Source Code Analysis

Source code package path:
`~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains`

The system provides two independent action execution service sides for different hardware configurations, and adaptively switches through the environment variable `CAMERA_TYPE` (`nuwa` or `usb`):

- **3D depth camera scheme** (default): Provided by `action_service_nuwa.py`, using the 3D point cloud and depth map to achieve physical-space back-projection inverse-solution grasping;
- **USB monocular camera scheme**: Provided by `action_service_usb.py`, combining visual servoing (Visual Servoing) and KCF tracking for closed-loop approach grasping.

4.1 Scheme One: 3D Depth Camera Spatial Inverse-Solution Grasping (action_service_nuwa.py)

In the depth camera scheme, the image captured by the camera is overlaid with a pixel grid, and the multimodal large model outputs a 2D bounding box `[x1, y1, x2, y2]`:

In the `ActionController.pick()` method, the process of solving from 2D pixels to the real physical coordinates of the robotic arm is completed:

```
def pick(self, x1, y1, x2, y2):
    # 1. Normalize the pixel bounding box to prevent coordinates from going out of bounds
```

```
    x1, y1, x2, y2 = self.normalize_bbox(x1, y1, x2, y2, 640, 480)

    # 2. Extract the valid foreground distance Z of the target bounding box in the depth map
    depth = self.get_dist(int((x1 + x2) / 2.0), int((y1 + y2) / 2.0))

    # 3. Combine with camera intrinsics to back-project into a 3D point in the camera coordinate frame (cam_x, cam_y, cam_z)
    point_cam = self.pixel_to_3d_ray(u_c, v_c, depth)

    # 4. Use TF transforms to convert the camera point into the robotic arm base coordinate frame (arm_base_link)
    point_arm = self.tf_buffer.transform(point_cam, "arm_base_link")

    # 5. Call the 6-DOF robotic arm inverse kinematics (IK) to solve the target servo angle for each joint
    target_angles = self.solve_ik(point_arm.point.x, point_arm.point.y, point_arm.point.z)

    # 6. Safely drive the servos downward and close the gripper
    self.pubSix_Arm(target_angles, runtime=2000)
    self.pubSingle_Arm(joint_id=6, joint_angle=30, runtime=1000)  # Close the gripper
```

Spatial relative placement (place)

When the large model understands "put it to the right/left of a certain object", the decision hub first generates the target placement point `(place_x, place_y, place_z)` based on the reference target's BBox and relative orientation, and `ActionController.place()` executes:

1. The robotic arm raises the held object to a safe transfer height;
2. The base rotates to the lateral unloading point in the target direction;
3. It slowly descends and opens the gripper to release the object;
4. The robotic arm withdraws to the standby pose to ensure it does not scrape or collide with the placed item.

4.2 Scheme Two: USB Monocular Camera Visual Servoing Grasping (action_service_usb.py)

A monocular USB camera cannot directly read the 3D physical depth of a pixel. Therefore, `action_service_usb.py` adopts a **closed-loop guided grasping scheme based on visual servoing (Visual Servoing)**:

```
def grasp_obj(self, x1, y1, x2, y2) -> bool | None:
    """ Start tracking and grasp the target based on the 640x480 BBox given by the large model """
    normalized = self.normalize_bbox(x1, y1, x2, y2)
    if normalized is None:
        self.get_logger().error("[BBOX] 归一化边界框失败")
        return False
    x1, y1, x2, y2 = normalized
    self.save_bbox_result(normalized, "grasp_obj")
```

```
    # Start the visual detection and servo tracking child node
    self.grasp_obj_future = Future()
    command = [
        "ros2", "run", "dofbot_pro_usb_voice", "KCF_detect",
        "--ros-args",
        "-p", f"x1:={x1}",
        "-p", f"y1:={y1}",
        "-p", f"x2:={x2}",
        "-p", f"y2:={y2}",
    ]
    return self._run_usb_grasp_action(
        "grasp_obj",
        self.grasp_obj_future,
        command,
        cancel_after=False,
    )
```

- **Closed-loop servo flow**:
    1. After receiving the object pixel box returned by the large model, the system starts the `KCF_detect` visual tracking node to lock onto the target;
    2. Under the closed-loop control of visual feedback, the robotic arm and the car chassis dynamically fine-tune their pose and distance, keeping the target object centered and approaching the optimal grasping working range of the robotic arm;
    3. After ranging and alignment converge, the grasping action sequence is triggered to descend and close the gripper, completing highly robust grasping under a monocular camera.

4.3 Tag Elevation Perception and Sorting (apriltag_remove_higher)

Both schemes support the tag recognition and sorting action `apriltag_remove_higher(target_high)`:

- Traverse all AprilTag machine-code tags detected in the field of view;
- Extract the vertical height of each tag relative to the reference plane;
- When a tag height greater than the set threshold (such as `0.05m`) is detected, it automatically triggers spatial alignment grasping and moves it to the side, realizing intelligent visual elevation sorting.